

DOCUMENTATION

Xeno FDE Internship Assignment – Shopify Data Ingestion & Insights Service

1. Problem Overview

The goal of this assignment is to build a **multi-tenant Shopify data ingestion and analytics platform** that simulates how Xeno helps enterprise retailers onboard, integrate, and analyze their customer data.

The system connects with multiple Shopify stores, ingests business-critical data (customers, orders, products, events), stores it in a relational database, and provides a dashboard to visualize performance metrics such as revenue trends, customer behavior, and temporal order distribution.

This solution is designed with **real-world constraints** in mind: multi-tenancy, data isolation, periodic data sync, extensibility, and clear observability.

2. Assumptions Made

1. Shopify Authentication

- For this assignment, stores are connected using **Admin API Access Tokens** (generated via custom/legacy apps).
- Full OAuth-based public Shopify app flow is out of scope but outlined in future improvements.

2. Multi-Tenancy Model

- Each Shopify store is treated as a **tenant**.
- Tenant data is always accessed using a tenantId to ensure isolation.
- No cross-store data access is allowed.

3. Data Freshness

- Shopify data is synced:
 - Manually via a “Sync” action in the dashboard.
 - Automatically every 30 seconds using a scheduler.
- Near real-time syncing via webhooks is supported for events.

4. Data Scope

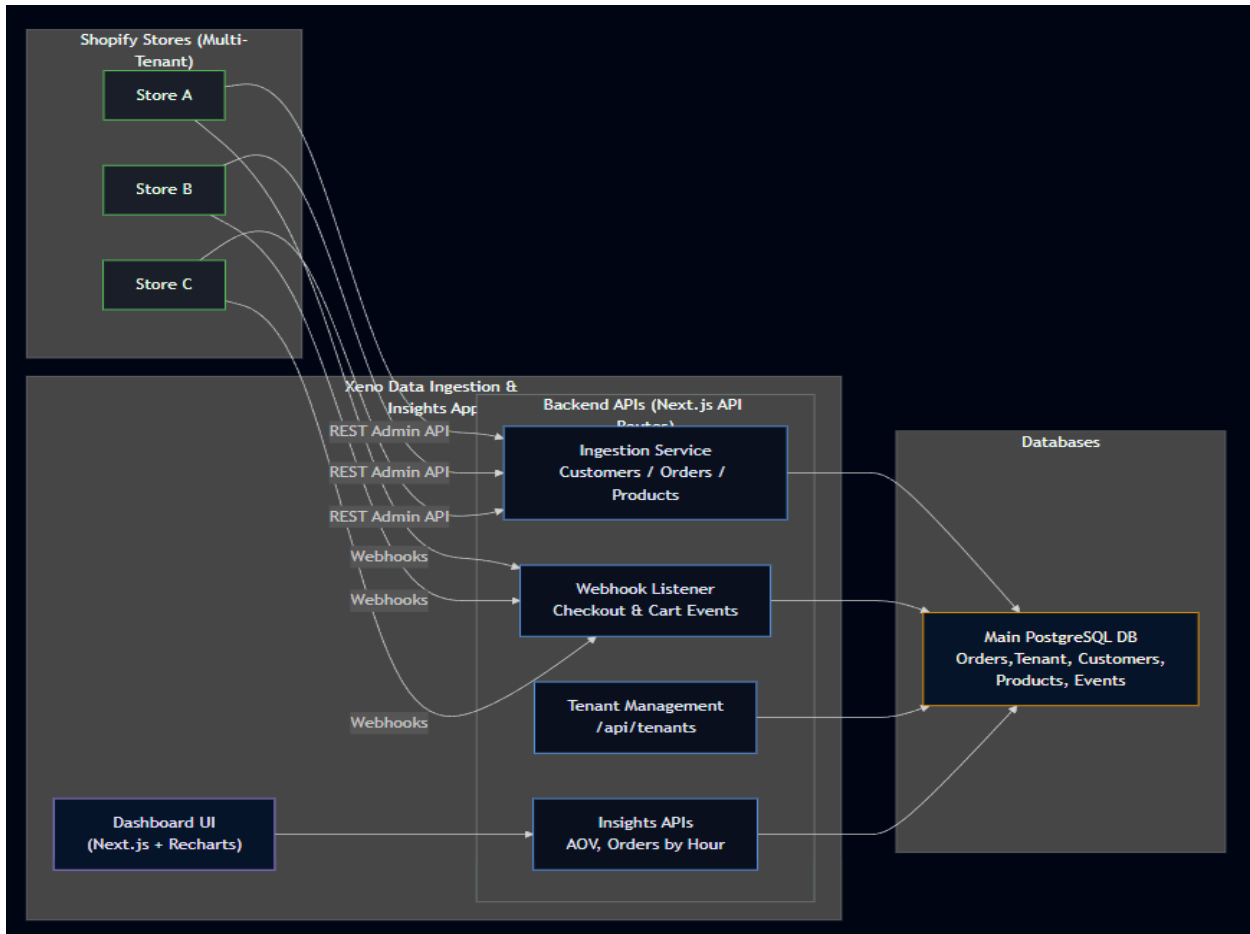
- Core entities ingested:
 - Customers
 - Orders
 - Products
- Historical data ingestion starts from the store installation time.

5. Users

- Email-based authentication is sufficient for this demo.
- One user can manage multiple stores.

3. High-Level Architecture

System Components



4. APIs Used

Shopify APIs

- Admin REST API
 - /admin/api/2024-01/customers.json
 - /admin/api/2024-01/orders.json
 - /admin/api/2024-01/products.json

Internal APIs (Next.js)

Endpoint	Method	Purpose
/api/tenants	POST	Register a new store

/api/tenants/:shop/sync	POST	Trigger data ingestion
/api/insights/summary	GET	Revenue, orders, customers
/api/insights/aov	GET	Average Order Value trend
/api/insights/orders-by-hour	GET	Orders distribution by hour
/api/shopfiy	POST	Webhook receiver

5. Data Models Used (PostgreSQL)

Tenant (isolated database)	Core Entities (main database)	Product	Event	Order
Tenant		- id	- id	- id
- id	Customer	- tenantId	- tenantId	- tenantId
- shop	- id	- shopifyId	- topic	- shopifyId
- accessToken	- tenantId	- title	- payload (JSON)	- totalPrice
- scope	- shopifyId	- price	- receivedAt	- currency
- installedAt	- firstName			- createdAt
	- lastName			- customerId
	- email			
	- totalSpent			

Each model includes tenantId for strict isolation.

6. Next Steps to Productionize

1. **OAuth-based Shopify App**
 - Replace token-based setup with OAuth install flow.
 - Publish app in Shopify App Store (private/public).
2. **Webhook Reliability**
 - Add HMAC verification.
 - Introduce retry mechanism using a queue (RabbitMQ / SQS).
3. **Scalability**
 - Move heavy ingestion to async workers.
 - Introduce Redis caching for dashboards.
4. **Observability**
 - Add ingestion logs and error metrics.
 - Track event ingestion lag.
5. **Data Warehousing**
 - Stream data to BigQuery / Snowflake for advanced analytics.
6. **Access Control**
 - Role-based access for multiple users per tenant.